

01 - PYTHON VARIABLES

Name: Vamshi | Date: 05/08/2026

- Variable is a name assigned to a value. This name refers to a memory location.
- Python variables are also called as objects.
- Syntax: **variable = value**

variable ----> name assigned for memory location & value -----> datatype value
stored in memory - variable

Rules to define a python variable name:

1. Variable names must **start with a letter or an underscore (_)**.
2. They **cannot start with a digit/number** but can end with digit/number.
3. It can contain **letters, digits, and underscores (_) but, only if above 2 rules are met.**
4. **Cannot contain spaces or special characters.**
5. They are **case-sensitive** (`python` , `Python` , and `PYTHON` are different variables).
6. It **cannot be Python keywords**. =====> Keywords are also known as reserved words.

```
In [5]: n=10 # Integer value
n
```

Out[5]: 10

```
In [6]: 10=n # Not Valid - syntax error - Rule 2
```

```
Cell In[6], line 1
    10=n # Not Valid - syntax error
    ^
SyntaxError: cannot assign to literal here. Maybe you meant '==' instead of '='?
```

```
In [8]: name = 'stark' # String value
name
```

Out[8]: 'stark'

```
In [9]: NAME # Name Error - Rule 5, becoz variable NAME is not present (its case sensi
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[9], line 1
----> 1 NAME #

NameError: name 'NAME' is not defined
```

```
In [10]: 1a = 25 # Syntax Error - Rule 2
```

```
Cell In[10], line 1
1a = 25
^
SyntaxError: invalid decimal literal
```

```
In [11]: name@ = 'tony' # Syntax Error - Rule 4
name@
```

```
Cell In[11], line 1
1 name@ = 'tony' # Name Error - Rule 4
^
SyntaxError: invalid syntax
```

```
In [12]: name_1 = 'tony' # Rule 3
name_1
```

```
Out[12]: 'tony'
```

```
In [13]: for = 45 # Syntax Error - Rule 6 (kw cannot be used as variable name)
```

```
Cell In[13], line 1
1 for = 45 # Rule 6
^
SyntaxError: invalid syntax
```

```
In [15]: FOR = 45
FOR
```

```
Out[15]: 45
```

```
In [23]: val = 53.45 # Float value
val
```

```
Out[23]: 53.45
```

```
In [33]: val_1 = True # Boolean value
val_1
```

```
Out[33]: True
```

To display the output on the screen, `print()` function is used.

```
In [19]: print(FOR)
print(name_1)
print(name)
print(n)
```

```
45
tony
stark
10
```

******The `type()` function is used to identify the datatype of the variable******

```
In [35]: print(type(n))
print(type(name))
print(type(val))
print(type(val_1))
```

```
<class 'int'>
<class 'str'>
<class 'float'>
<class 'bool'>
```

To check the list of all keywords and print total number of keywords:

```
In [37]: import keyword
keyword.kwlist # Lists all the key words
```

```
Out[37]: ['False',
          'None',
          'True',
          'and',
          'as',
          'assert',
          'async',
          'await',
          'break',
          'class',
          'continue',
          'def',
          'del',
          'elif',
          'else',
          'except',
          'finally',
          'for',
          'from',
          'global',
          'if',
          'import',
          'in',
          'is',
          'lambda',
          'nonlocal',
          'not',
          'or',
          'pass',
          'raise',
          'return',
          'try',
          'while',
          'with',
          'yield']
```

```
In [38]: kw = len(keyword.kwlist)
print("The total number of keywords are", kw)
```

The total number of keywords are 35